AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 7

ทนทานต่อความผิดพลาด · ฐานข้อมูลล่มตอบ 503 · กู้งานที่ค้างเมื่อ worker หยุดกลางคัน triage ผลสแกนของตัวเอง · รันทั้งระบบด้วย `docker compose` คำสั่งเดียว
ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 6"

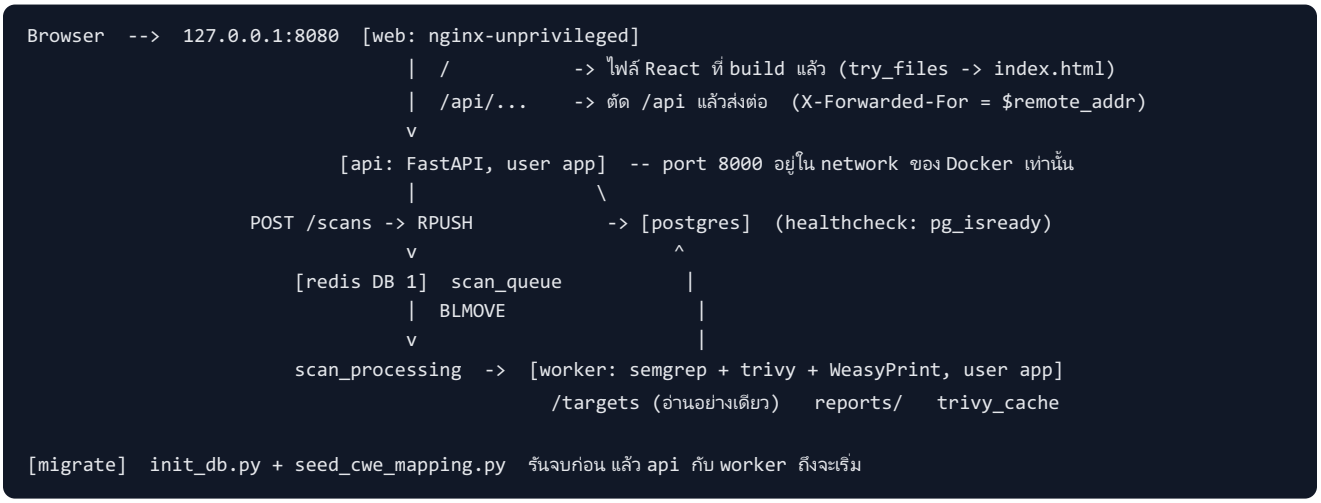
> ชื่อระบบ: **VulnCompass**
> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 16–19 ตุลาคม 2026 · 4 commit · test 102 → **110** ตัว

1. สรุปสั้นๆ

ขั้นที่ 6 ทำให้ระบบใช้งานได้ครบ 3 ทาง (CLI, CI/CD, หน้าเว็บ) ขั้นที่ 7 ทำให้ระบบ**ไม่พังง่ายเมื่อมีอะไรผิดปกติ** และ**ให้คนอื่นรันได้ด้วยคำสั่งเดียว** ทุกข้อทดสอบกับของจริงด้วย ไม่ใช่แค่ test: ปิด Docker จริง, กด Ctrl+C ที่ worker จริง และสแกน NodeGoat ผ่าน Docker จริง

เรื่อง	ก่อน (จบขั้นที่ 6)	หลัง (จบขั้นที่ 7)
ฐานข้อมูลปิดหรือต่อไม่ได้	**500 Internal Server Error** เหมือนโค้ดมีบั๊ก	**503** "ฐานข้อมูลไม่พร้อมใช้งาน กรุณาลองใหม่ภายหลัง" และรายละเอียด error อยู่แค่ใน log ฝั่ง server
กด Ctrl+C ที่ worker ระหว่างสแกน	งาน**ค้างเป็น "running" ตลอดไป** ต้องแก้ฐานข้อมูลด้วยมือ	เปิด worker ใหม่แล้วงานจะ**เปลี่ยนเป็น failed ให้เอง** ส่วนงานที่ยังไม่ทันเริ่มจะกลับเข้าคิว
Semgrep สแกนตัวเอง	2 รายการ medium ยังไม่ได้ตรวจ	ตรวจแล้วว่าไม่ใช่ช่องโหว่ทั้งคู่ · บังคับ autoescape · มี test กัน XSS · **0 รายการ**
วิธีรันระบบ	เปิดเอง 5 อย่าง และต้องติดตั้ง Python, Node, semgrep, trivy ในเครื่อง	`docker compose --profile app up -d` แล้วเปิด `localhost:8080`
สิทธิ์ของโปรแกรมใน Docker	-	รันเป็น user `app` ไม่ใช่ root · API ไม่เปิด port ออกมาที่เครื่อง · โฟลเดอร์ที่สแกนเปิดแบบอ่านอย่างเดียว
จำนวน test	102	110

ภาพรวมสถาปัตยกรรมเมื่อรันใน Docker



Postgres และ Redis ยังเปิดได้แบบเดิมด้วย `docker compose up -d` (ใช้ตอนพัฒนา) ส่วน 4 service ใหม่อยู่ใน profile `app` จะเปิดเฉพาะตอนสั่ง `--profile app`

2. C7a: ฐานข้อมูลล่ม ตอบ 503 แทน 500

1 ปัญหา

ถ้า Docker ปิดอยู่ ทุก endpoint ที่แตะฐานข้อมูลจะตอบ `500 Internal Server Error` ซึ่งแปลว่า "โค้ดมีบั๊ก"
ทั้งที่จริงเป็นแค่บริการที่ยังไม่พร้อม คนดูแลระบบเลยเข้าใจผิดว่าต้องไปแก้โค้ด และ monitoring หรือ load balancer ก็นับเป็น error ของแอป

	500 (เดิม)	503 (ใหม่)
ความหมาย	server มีบั๊ก	บริการยังไม่พร้อม ลองใหม่ได้
คนดูแลระบบต้อง	ไปแก้โค้ด	ไปเปิดฐานข้อมูล
ผู้ใช้เห็น	`Internal Server Error`	ฐานข้อมูลไม่พร้อมใช้งาน กรุณาลองใหม่ภายหลัง

2 วิธีแก้: exception handler จุดเดียว

แทนที่จะใส่ `try/except` ในทุก endpoint (มีมากกว่า 10 ตัว) เราลงทะเบียน handler กับ FastAPI ไว้ที่เดียว ถ้า endpoint ไหนเจอ `OperationalError` (error ของ SQLAlchemy ตอน**ต่อ**ฐานข้อมูลไม่ได้) จะมาจบที่ฟังก์ชันนี้

```
@app.exception_handler(OperationalError)
def database_unavailable(request: Request, exc: OperationalError):
    logger.error("ต่อฐานข้อมูลไม่ได้ (%s %s): %s", request.method, request.url.path, exc.orig)
    return JSONResponse(status_code=503, content={"detail": "ฐานข้อมูลไม่พร้อมใช้งาน กรุณาลองใหม่ภายหลัง"})
```

จุดที่ตั้งใจ	เหตุผล
จับแค่ `OperationalError`	error จาก SQL ที่เขียนผิดยังเป็นบั๊กจริง และควรเป็น 500 ต่อไป
🔒 ไม่ส่ง `exc` ให้ผู้ใช้	ข้อความ error จริงมี host, port และชื่อ user ของฐานข้อมูล ถ้าส่งไปจะเป็น **information disclosure** จึงบันทึกไว้ใน log ฝั่ง server เท่านั้น
logging ใช้ `%s`	สร้างข้อความเฉพาะตอนที่ต้องแสดงจริง เป็นแนวปฏิบัติมาตรฐานของ logging

3 ทดสอบ

test: `app.dependency_overrides[get_session] = BrokenSession` ใช้ฐานข้อมูลปลอมที่ทุกคำสั่ง raise `OperationalError` (ผ่าน `__getattr__`) จึงไม่ต้องปิด Docker จริงก็ทดสอบได้ และรันบน GitHub ได้ test มี 2 ข้อ: login ได้ 503 และข้อความตอบกลับไม่มี `127.0.0.1` ทั้ง 2 ข้อ**พังกับโค้ดเดิม**ตามที่ควรเป็น

ของจริง: ปิด Docker แล้วลอง login ได้ 503 พร้อมข้อความภาษาไทย พอเปิด Docker กลับมา หน้าเว็บใช้งานได้ทันทีโดยไม่ต้อง restart uvicorn

ไฟล์: `main.py`, `tests/test_db_unavailable.py` (ใหม่) · commit `0f7471b`

3. C7b: worker กู้งานที่ค้างเมื่อหยุดกลางคัน

4 ปัญหา

worker เดิมใช้ `BLPOP` ซึ่ง**ดึงงานออกจาก Redis ทันที** ถ้าระหว่างสแกนมีคนกด Ctrl+C, ปิดหน้าต่าง หรือเครื่องดับ ฐานข้อมูลจะยังเขียนว่างานเป็น `running` ไปตลอด และใน Redis ก็ไม่เหลืออะไรให้รู้ว่างานไหนทำค้างไว้ (ในขั้นที่ 6 ต้องแก้งานค้าง 9 งานในฐานข้อมูลด้วยมือ ซึ่งเป็นหนี้เทคนิคที่ยกมาแก้ในรอบนี้)

5 วิธีแก้: Reliable Queue (รูปแบบมาตรฐานของ Redis)

	เดิม `BLPOP`	ใหม่ `BLMOVE`
หยิบงาน	ดึงออกจากคิว แล้วงานก็หายไป	**ย้าย**จาก `scan_queue` ไปที่ `scan_processing` ในคำสั่งเดียว (atomic)
ทำเสร็จ	-	`LREM` ลบออกจาก `scan_processing` (`finish_scan`)
worker ดับกลางคัน	งานหาย	งานยังค้างอยู่ใน `scan_processing`
เปิด worker ใหม่	ไม่รู้อะไรเลย	ตรวจ `scan_processing` ก่อนรับงานใหม่

ตอนเปิด worker งานที่ค้างอยู่จะถูกจัดการตามสถานะในฐานข้อมูล:

สถานะ	แปลว่า	จัดการ
`running`	สแกนไปครึ่งทาง ผลไม่ครบ	เปลี่ยนเป็น `failed` และ ScanRun ที่ค้างจะได้ error_message "worker หยุดทำงานระหว่างสแกน"
`queued`	หยิบไปแล้วแต่ยังไม่ทันเริ่ม	ใส่กลับเข้าคิว
`completed` / `failed`	ทำเสร็จแล้วแต่ยังไม่ทันลบออก	แค่ลบออกจาก `scan_processing`

ทำไมต้อง atomic: ถ้าแยกเป็น pop แล้วค่อย push ทีละคำสั่ง แล้ว worker ดับตรงกลางพอดี งานก็จะหายเหมือนเดิม **ทำไมกด Ctrl+C แล้วงานยังค้าง:** `process_task` ดักแค่ `Exception` แต่ Ctrl+C เป็น `KeyboardInterrupt` จึงไม่ถึงบรรทัด `finish_scan` เราตั้งใจให้เป็นแบบนี้ เพื่อให้รอบหน้ามาเก็บ

> ออกแบบสำหรับ **worker ตัวเดียว** ถ้าจะมีหลายตัว ต้องแยกรายการ processing ตามชื่อ worker ไม่อย่างนั้นตัวที่เพิ่งเปิดจะไปเก็บงานที่ตัวอื่นกำลังทำอยู่ ข้อจำกัดนี้เขียนไว้ใน comment ของโค้ดแล้ว

6 ทดสอบ

test 4 ข้อ: `FakeRedis` (แยกเป็น `tests/fake_redis.py` ให้ 2 ไฟล์ใช้ร่วมกัน) จำลอง `BLMOVE` / `LREM` / `LRANGE` และ `FakeSession` จำลองฐานข้อมูลแค่ 3 คำสั่งที่ใช้ จึงรันบน GitHub ได้โดยไม่ต้องมี Postgres

ของจริง: สั่งสแกนงาน #46 แล้วกด Ctrl+C ตอน semgrep เริ่ม จากนั้นเปิด worker ใหม่ ได้ `ScanJob 46 ค้างเป็น running จาก worker ตัวก่อน เปลี่ยนเป็น failed` ตรวจในฐานข้อมูลแล้ว: งาน = failed, ScanRun 98 = failed พร้อมเหตุผล และ `scan_processing` ว่าง

ไฟล์: `scan_queue.py`, `worker.py`, `tests/fake_redis.py` (ใหม่), `tests/test_scan_queue.py`, `tests/test_worker_recovery.py` (ใหม่) · commit `70c9645`

4. C7c: triage ผลสแกน Semgrep ของตัวเอง

Triage คือการตัดสินว่าคำเตือนแต่ละข้อเป็นช่องโหว่จริงไหม ไม่ใช่การทำให้คำเตือนหายไป เมื่อตัดสินแล้วต้องเขียนเหตุผลไว้ในโค้ด และถ้าทำได้ควรมี test ที่พิสูจน์ผลการตัดสินนั้นด้วย

	1. jinja2 (`generate_report.py`)	**2. urllib (`epss.py`)**
กฎ	`direct-use-of-jinja2`	`dynamic-urllib-use-detected`
Semgrep บอกว่า	ใช้ jinja2 ตรงๆ อาจไม่ escape HTML ทำให้เกิด XSS	urllib รองรับ `file://` ถ้าคนนอกกำหนด URL ได้จะอ่านไฟล์ในเครื่องได้
ตรวจแล้วพบว่า	เปิด escape อยู่แล้ว (template ทุกตัวเป็น `.html`) และกฎนี้เขียนไว้สำหรับ Flask ซึ่งเราไม่ได้ใช้	URL ขึ้นต้นด้วย `https://api.first.org/...` ที่เขียนตายตัวไว้ ส่วน CVE ผ่าน `urlencode` เข้าไปอยู่ใน query เท่านั้น
ผล	**ไม่ใช่ช่องโหว่**	**ไม่ใช่ช่องโหว่**
สิ่งที่ทำ	เปลี่ยนเป็น `autoescape=True` (escape ทุกไฟล์ ไม่ขึ้นกับนามสกุล) + **test พิสูจน์** + `nosemgrep` พร้อมเหตุผล	`nosemgrep` พร้อม comment เหตุผล

7 ทำไมข้อ 1 สำคัญกว่าที่เห็น: Stored XSS ผ่านรายงาน

ชื่อช่องโหว่และ path ใน dashboard มาจาก**โค้ดของคนอื่นที่เราสแกน** ถ้ามีคนตั้งชื่อ package หรือเขียน rule ให้มี `<script>` แล้วรายงานไม่ escape คนที่เปิด dashboard ก็จะโดนรันสคริปต์ทันที เครื่องมือ security ที่โดนเจาะผ่านรายงานของตัวเองจะเสียความน่าเชื่อถือมาก

ลองปิด autoescape ดูแล้ว `<script>` หลุดเข้าไปใน HTML จริง จึงเขียน `test_report_escaping.py` ใช้ `@pytest.mark.parametrize` รันกับทั้ง `report_template.html` และ `dashboard_template.html` และตรวจทั้ง 2 ทาง: ของอันตรายต้องไม่อยู่ใน HTML และชื่อที่ escape แล้ว (`<script>`) ต้องยังแสดงอยู่

8 ทำไมใช้ `# nosemgrep: <ชื่อกฎ>` แบบระบุชื่อ

ถ้าเขียนแค่ `# nosemgrep` จะปิดทุกกฎในบรรทัดนั้น ถ้าวันหลังมีกฎอื่นเจอปัญหาจริงในบรรทัดเดียวกันก็จะเงียบไปด้วย การระบุชื่อกฎทำให้ปิดเฉพาะคำเตือนที่ตรวจแล้ว ส่วน comment ข้างบนบอกคนที่มาอ่านทีหลังว่า**ทำไม**ถึงปิด

ผล: รัน Semgrep กับ 2 ไฟล์นี้ในเครื่อง ได้ **0 รายการ** (เดิม 2) · test 108 → 110

ไฟล์: `generate_report.py` , `epss.py` , `tests/test_report_escaping.py` (ใหม่) · commit `f620802`

5. C7d: รันทั้งระบบด้วย Docker Compose

9 ไฟล์ใหม่และหน้าที่

ไฟล์	หน้าที่
`Dockerfile`	image เดียวใช้ทั้ง API และ worker: Python 3.14.8, Pango + ฟอนต์ไทย (`fonts-thai-tlwg`), Trivy จาก `install_trivy.sh` (ตรวจ SHA-256), semgrep
`.dockerignore`	🔓 กัน `.env` และ `ca-bundle.pem` ไม่ให้ฝังเข้าไปใน image รวมถึงตัด venv, tests, reports ออก
`frontend/Dockerfile`	multi-stage: Node 24.21 build React แล้วเอาแค่ `dist/` ไปใส่ nginx (ไม่มี Node และ `node_modules` ใน image สุดท้าย)
`frontend/nginx.conf`	เสิร์ฟหน้าเว็บ ส่ง `/api/` ต่อให้ FastAPI และใช้ `try_files` ให้กด F5 ที่ `/scans` ได้
`docker-compose.yml`	เพิ่ม `migrate` , `api` , `worker` , `web` ใน profile `app` และ healthcheck ให้ Postgres

10 การตัดสินใจด้าน security

เรื่อง	ทำอะไร และทำไม
ไม่รันเป็น root	`USER app` และใช้ `nginx-unprivileged` ถ้ามีคนเจาะเข้ามาได้ จะได้แค่สิทธิ์ของ user นี้
ไม่เปิด port ของ API	uvicorn ฟังที่ `0.0.0.0` ใน container (จำเป็น เพราะ nginx อยู่คนละ container) แต่ไม่ได้ map port 8000 ออกมาที่เครื่อง จึงเข้าได้ผ่าน nginx เท่านั้น ส่วน port เดียวที่เปิดคือ `127.0.0.1:8080`
IP จริงของผู้ใช้	หลัง proxy ทุก request จะมาจาก IP ของ nginx ทำให้ login ผิด 5 ครั้งล็อกทุกคนพร้อมกัน จึงใช้ `--proxy-headers` ให้ uvicorn อ่าน IP จริงที่ nginx ส่งมา
กันการปลอม IP	ใช้ `X-Forwarded-For $remote_addr` แทน `$proxy_add_x_forwarded_for` ที่เห็นบ่อยในตัวอย่างทั่วไป เพราะแบบหลังจะต่อท้ายค่าที่ผู้ใช้ส่งมาเอง ทำให้ผู้โจมตีเปลี่ยน IP ปลอมได้ทุกครั้งที่เดารหัสผ่าน แล้วหลบ rate limit ได้
โฟลเดอร์ที่สแกน	mount `../Targets` ไปที่ `/targets:ro` (อ่านอย่างเดียว) และตั้ง `SCAN_ALLOWED_REPO_ROOTS=/targets` ลองสั่งสแกน `/app` แล้วถูกปฏิเสธ 400
ไม่เปิด `docker.sock`	ถ้าเปิด DAST (ZAP) จะใช้ใน Docker ได้ แต่ `docker.sock` **เท่ากับสิทธิ์ root ทั้งเครื่อง** จึงตั้งใจไม่ทำ ถ้าต้องการ DAST ให้ใช้ worker ในเครื่องแทน
ล็อกเวอร์ชัน image	`python:3.14.8-slim` , `node:24.21.0-alpine` , `nginx-unprivileged:1.28.2-alpine` ทุกตัวออกมาเกิน 2 สัปดาห์ (Node 24.22.0 ออกมาได้แค่ 5 วันจึงไม่ใช้)

11 การตัดสินใจด้านการใช้งาน

เรื่อง	เหตุผล
profile `app`	`docker compose up -d` ยังเปิดแค่ Postgres กับ Redis เหมือนเดิม ขั้นตอนพัฒนาประจำวันจึงไม่เปลี่ยน
`x-backend: &backend`	YAML anchor เขียนค่าที่ใช้ร่วมกันไว้ครั้งเดียว แล้วใช้ใน migrate, api และ worker ด้วย `<<: *backend`
healthcheck + `service_healthy`	รอจน Postgres **พร้อมรับคำสั่ง** ไม่ใช่แค่รอให้ container เปิด
`service_completed_successfully`	api และ worker เริ่มหลัง migrate จบโดยไม่มี error เท่านั้น
ทับค่าจาก `.env`	`.env` ใช้ `localhost` และ path ของ Windows (เช่นไฟล์ CA ของ Norton) จึงทับด้วยชื่อ service และ CA ของ Linux ใน `environment:`
Redis DB 1	แยกคิวจากตอนรันในเครื่อง (DB 0) ดูปัญหาข้อ 6.4
layer cache	copy `requirements.txt` แล้ว `pip install` ก่อนค่อย `COPY . .` ถ้าแก้แค่โค้ด build ใหม่ไม่กี่วินาที แทนที่จะเป็นหลายนาที
`trivy_cache` volume	ฐานข้อมูลช่องโหว่ของ Trivy มีขนาด 128 MB (โหลดครั้งแรก 1 นาที 26 วินาที) เก็บไว้ ไม่ต้องโหลดใหม่ทุกครั้ง
`TZ=Asia/Bangkok`	container ใช้เวลา UTC เป็นค่าเริ่มต้น วันที่ในรายงานเลยคลาดไป 7 ชั่วโมง

12 ทดสอบ

ชุดทดสอบแยก (project `vctest`, ฐานข้อมูลใหม่, `.env` ชั่วคราว) ใช้ `!reset` / `!override` ของ compose เพื่อเปลี่ยนชื่อ container และ port ไม่ให้ชนกับของจริง ผล: migrate สร้างตาราง + seed 53 CWE · login ผ่าน nginx · สแกน NodeGoat ได้ 127 findings · ดาวน์โหลด PDF 25 หน้า ภาษาไทยถูกต้อง · สแกน container image `alpine:3.18` ได้โดยไม่ต้องใช้ Docker ในเครื่อง · สแกน `/app` ถูกปฏิเสธ จากนั้นลบชุดทดสอบทิ้งทั้งหมด

ของจริง: `migrate` เจอฐานข้อมูลเดิมแล้วรัน migration (ข้อมูลอยู่ครบ) · งาน #49 (SAST + Dependency ที่ `/targets/NodeGoat`) เสร็จทั้งคู่ และมีลิงก์รายงานครบ

ไฟล์: `Dockerfile`, `.dockerignore`, `frontend/Dockerfile`, `frontend/.dockerignore`, `frontend/nginx.conf` (ใหม่ทั้งหมด), `docker-compose.yml` · commit `a03431e`

ข้อจำกัดที่รู้อยู่แล้ว: (1) DAST ใช้ใน Docker ไม่ได้ตามเหตุผลเรื่อง `docker.sock` ด้านบน (2) รายงานเก่าที่สร้างตอนรันบน Windows เก็บ path แบบ `reports\...` ซึ่ง Linux อ่านไม่ออก จึงดาวน์โหลดผ่าน `:8080` ไม่ได้ (รายงานใหม่ใช้ได้ปกติ) อยู่ในรายการ C7 ◍

6. ปัญหาที่เจอระหว่างทาง และบทเรียน

6.1 ข้อความจากคำอธิบายติดเข้าไปในไฟล์ test

อาการ: `SyntaxError: invalid character '—' (U+2500)` ที่บรรทัด 39 ของ `test_db_unavailable.py`
สาเหตุ: ตอน copy โค้ด ข้อความ "กด Ctrl+S" ตารางคำอธิบาย และเส้นคั่นติดเข้าไปด้วย
แก้: ลบตั้งแต่บรรทัด 34 ถึงท้ายไฟล์ (`Ctrl+Shift+End`)
บทเรียน: copy เฉพาะในกล่องโค้ด และอ่าน error ให้ดู "บรรทัดที่" กับตัวอักษรที่ Python บอก

6.2 `import secrets` หายไปตอนรวม import

อาการ: แก้ส่วนบนของ `main.py` เป็น `import os, logging, redis` แล้ว `import secrets` หายไป แต่โค้ดยังเรียก `secrets.token_urlsafe` และ `secrets.compare_digest` อยู่
ทำไมอันตราย: pytest ยังไม่เจอ เพราะหยุดที่ error ข้อ 6.1 ไปก่อน ถ้า commit ไปแบบนี้ API จะเปิดไม่ขึ้น (`NameError`)
บทเรียน: ดู `git diff` ก่อน commit เสมอ บรรทัด `-` ที่ไม่ได้ตั้งใจลบเป็นสัญญาณเตือน · PEP 8 ให้ import บรรทัดละ 1 module ซึ่งทำให้ diff อ่านง่ายและพลาดยากกว่า

6.3 วาง Dockerfile สลับไฟล์

อาการ: `target web: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory`
สาเหตุ: โค้ดของ `frontend/Dockerfile` ไปอยู่ใน `frontend/.dockerignore` (605 bytes แทนที่จะเป็น 18 bytes)
บทเรียน: ดูชื่อแท็บก่อนกด Ctrl+S · ขนาดไฟล์ที่ผิดปกติช่วยหาจุดผิดได้เร็ว

6.4 สองระบบแย่งคิวเดียวกัน (งาน #47) บั๊กจริงที่ test ไม่เจอ

อาการ: งาน #47 failed: `lstat C:\Allprojects\Targets\NodeGoat: no such file or directory` ทั้งที่หน้าเว็บ `:8080` ปฏิเสธ path `C:\...` ไปแล้ว
สาเหตุ: งานมาจากหน้าเว็บ `:5173` (uvicorn บน Windows ซึ่งยอมรับ `C:\...`) แต่ทั้งสองระบบใช้ Redis DB 0 เดียวกัน worker ใน Docker เลยหยิบงานของฝั่ง Windows ไปทำ กลับกันก็เป็นได้เช่นกัน
แก้: ให้ฝั่ง Docker ใช้ `redis://redis:6379/1` ทำให้ใช้ Postgres ร่วมกัน (เห็นประวัติเดียวกัน) แต่คิวแยกกัน
บทเรียน: test ทุกข้อผ่าน แต่บั๊กนี้โผล่เฉพาะตอนรันของจริงแบบ 2 ระบบพร้อมกัน จึงต้องลองระบบจริงด้วยเสมอ · ทรัพยากรที่ใช้ร่วมกัน (คิว, ไฟล์, ฐานข้อมูล) ต้องคิดว่าใครอ่านและใครเขียน

6.5 ปัญหาที่เจอตอนเตรียมในชุดทดสอบ (แก้ก่อนสอน)

อาการ	สาเหตุและวิธีแก้
API ตอบ "ไม่พบโฟลเดอร์ repo_path"	API ตรวจว่าโฟลเดอร์มีอยู่จริงก่อนใส่คิว จึงต้อง mount `/targets` ให้ api ด้วย ไม่ใช่แค่ worker
เข้าเว็บที่ port ทดสอบไม่ได้ (000)	`ports: !reset [...]` ล้างค่าทิ้งทั้งหมดรวมถึงค่าใหม่ด้วย ต้องใช้ `!override`
Trivy เขียน cache ไม่ได้ (คาดไว้ก่อน)	volume ใหม่จะได้ owner เป็น root ถ้าโฟลเดอร์ยังไม่มีใน image จึงสร้าง `/home/app/.cache/trivy` และ `chown` ใน Dockerfile
Semgrep ต่อ `semgrep.dev` ไม่ได้ (SSL) ในเครื่อง	Norton ดักตรวจ HTTPS บน Windows ส่วนใน container ทดสอบแล้วต่อได้ตรงๆ จึงใช้ CA ของ Linux ใน Docker

7. สถานะงาน และสิ่งที่ยังเหลือ

commit ในรอบนี้ (ทั้งหมดอยู่บน `main` และ push แล้ว)

```
0f7471b  C7a  Return 503 without internal details when the database is unavailable
70c9645  C7b  Recover scans left unfinished when the worker stops mid-scan
f620802  C7c  Triage Semgrep findings: always autoescape reports and document the fixed EPSS URL
a03431e  C7d  Run the whole system with docker compose: API, worker, and nginx web container
```

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`main.py`	แก้: exception handler 503
`scan_queue.py`, `worker.py`	แก้: BLMOVE, finish_scan, กู้งานตอนเริ่ม
`generate_report.py`, `epss.py`	แก้: autoescape=True, nosemgrep พร้อมเหตุผล
`Dockerfile`, `.dockerignore`, `frontend/Dockerfile`, `frontend/.dockerignore`, `frontend/nginx.conf`	ใหม่
`docker-compose.yml`	แก้: 4 service ใหม่, healthcheck, profile, Redis DB 1
`tests/`	ใหม่: test_db_unavailable, fake_redis, test_worker_recovery, test_report_escaping · แก้: test_scan_queue · รวม **110 test**

สถานะ C7

ระดับ	งาน	สถานะ
🟢	ฐานข้อมูลล่มตอบ 503	**เสร็จ**
🟢	กู้งานที่ค้าง running	**เสร็จ**
🟢	triage Semgrep 2 รายการ	**เสร็จ**
🟢	Docker ทั้งระบบ	**เสร็จ**
⚪	path รายงานเก่า (`\` กับ `/`) · `datetime.utcnow` ที่ deprecated · คำเตือน httpx ใน test	ยังเหลือ
⚪	รหัสผ่านให้ Redis · login limiter ที่ยังอยู่ได้หลัง restart · session ฝั่ง server	ยังเหลือ
⚪	ใช้ path แบบ absolute ให้ reports · เปลี่ยนชื่อ `is_repo_allowed` → `is_path_inside` · ค้นหาแบบ full-text · favicon แทนโลโก้ Vite	ยังเหลือ
⚪	DAST ใน Docker แบบปลอดภัย (ZAP เป็น service แยกที่เรียกผ่าน API แทนการใช้ docker.sock)	ไอเดียต่อยอด

ขั้นต่อไป: C8

README (วิธีรันด้วยคำสั่งเดียว), ภาพหน้าจอ, diagram สถาปัตยกรรม, threat model (รวมเรื่องรหัสผ่านหลุดในขั้นที่ 6 และการตัดสินใจเรื่อง `docker.sock` ในขั้นนี้) และวิดีโอ demo

8. สรุปให้อ่าน: รอบนี้ทำอะไร ทำไม และได้อะไร

ทำอะไร

รอบนี้ไม่ได้เพิ่มความสามารถใหม่ให้ผู้ใช้เห็นมากนัก แต่ทำให้ระบบ**รับมือกับความผิดพลาดได้** และ**พร้อมให้คนอื่นรัน**:
ฐานข้อมูลล่มแล้วตอบอย่างสุภาพโดยไม่เผยข้อมูลภายใน, worker ที่ถูกปิดกลางคันไม่ทิ้งงานค้างไว้, คำเตือนจาก Semgrep ถูกตรวจและพิสูจน์ด้วย test และทั้งระบบรันได้ด้วย `docker compose` คำสั่งเดียว โดยออกแบบให้ปลอดภัยตั้งแต่แรก

ทำไมต้องทำ

ความทนทาน: ระบบที่ทำงานได้เฉพาะตอนทุกอย่างปกติยังไม่ใช่ระบบที่ใช้งานจริงได้ ในการใช้งานจริง ฐานข้อมูลจะต้องมีวันล่ม และ process จะต้องมีวันถูกปิดกลางคัน
Triage: เครื่องมือ security ที่ปล่อยคำเตือนของตัวเองค้างไว้ หรือปิดคำเตือนโดยไม่มีเหตุผล จะไม่น่าเชื่อถือ
Docker: คนที่ดู portfolio ส่วนใหญ่จะไม่ติดตั้ง Python, Node, semgrep และ trivy เพื่อลองโปรเจกต์ แต่จะลองถ้าใช้แค่คำสั่งเดียว

ได้อะไร

ด้าน	ผลที่ได้
ระบบ	รันได้ 2 แบบ: พัฒนาในเครื่อง (`:5173`) หรือทั้งระบบใน Docker (`:8080`) ใช้ฐานข้อมูลร่วมกัน แต่คิวแยกกัน
ความปลอดภัยของระบบเอง	ไม่เผยรายละเอียดฐานข้อมูลใน error · autoescape ทุกรายงาน · non-root · API ไม่เปิด port · กันการปลอม IP · targets อ่านอย่างเดียว · `.env` ไม่อยู่ใน image · ไม่ใช้ docker.sock
คุณภาพ	test 102 → 110 · test ใหม่พังกับโค้ดเดิมจริง · ทุกข้อลองกับของจริงแล้ว · Semgrep 2 → 0
ทักษะ	FastAPI exception handler, Redis reliable queue (BLMOVE), test doubles (FakeRedis, FakeSession, dependency_overrides), Dockerfile (layer cache, non-root, multi-stage), Docker Compose (profile, healthcheck, anchor, depends_on condition), nginx reverse proxy

เรื่องที่เล่าตอนสัมภาษณ์งานได้

หัวข้อ	เล่าว่า
Graceful degradation	ฐานข้อมูลล่มต้องตอบ 503 ไม่ใช่ 500 → ใช้ exception handler จุดเดียวครอบทุก endpoint → ไม่ส่ง host/port ให้ผู้ใช้ (information disclosure) → test ด้วย dependency override แทนการปิด Docker
Reliable queue	BLPOP ทำให้งานหายเมื่อ worker ดับ → BLMOVE ไป processing list แบบ atomic → กู้งานตอนเริ่มตามสถานะในฐานข้อมูล → รู้ข้อจำกัดว่าออกแบบไว้สำหรับ worker ตัวเดียว
Triage ไม่ใช่การปิดเสียงเตือน	วิเคราะห์ว่า input มาจากไหน (ชื่อช่องโหว่มาจากโค้ดของคนอื่น = ข้อมูลที่ไว้ใจไม่ได้) → พิสูจน์ด้วย test ที่ลองปิด escape แล้วพังจริง → `nosemgrep` แบบระบุกฎพร้อมเหตุผล
Container security	non-root, ไม่ publish port ภายใน, read-only mount, ไม่ฝัง secret ใน image, ล็อกเวอร์ชัน image และ checksum ของ binary
Trade-off ที่ตั้งใจเลือก	ไม่ใช้ docker.sock แม้จะทำให้ DAST ใช้ใน Docker ไม่ได้ เพราะเท่ากับให้ root ทั้งเครื่อง → เขียนเป็นข้อจำกัดชัดๆ และเสนอทางที่ปลอดภัยกว่า (ZAP เป็น service แยก)
Reverse proxy กับ rate limit	หลัง proxy ทุกคนมี IP เดียวกัน → อ่าน IP จริงด้วย proxy headers → แต่ต้องไม่เชื่อ header ที่ผู้ใช้ส่งมาเอง (`$remote_addr` ไม่ใช่ `$proxy_add_x_forwarded_for`)
บั๊กที่ test ไม่เจอ	2 ระบบใช้คิวเดียวกันแล้วหยิบงานของกันและกัน → เจอตอนลองของจริง → แยกด้วย Redis DB

สรุปในประโยคเดียว: จบขั้นที่ 7 VulnCompass ไม่ได้แค่ทำงานได้ตอนทุกอย่างปกติ แต่รับมือกับฐานข้อมูลล่มและ worker ที่ถูกปิดกลางคันได้ ตรวจคำเตือนจากการสแกนตัวเองจนเหลือ 0 และรันทั้งระบบได้ด้วยคำสั่งเดียวโดยออกแบบให้ปลอดภัยตั้งแต่แรก ขั้นต่อไป (C8) คือทำให้คนที่เข้ามาดู repo เข้าใจโปรเจกต์ได้ภายในไม่กี่นาที